StockGo

Asignatura: Desarrollo Móvil Integral
Docente: Ing. Froylán Alanís
Integrantes: Alexis Jiménez Salas, Melani Judith Zermeño Yáñez
Grupo y Grado : 10 D
Fecha: 1 de noviembre de 2026

Indice

1. Descripción del proyecto 3
    1.1 Problemática 3
    1.2 Justificación 3
    1.3 Objetivo general 3
    1.4 Usuarios destinatarios 3
    1.5 Funcionalidades principales 3
2. Arquitectura 5
    2.1 Componentes 5
    2.2 Organización del código 5
    2.3 Flujo de información 6
3. Integración de servicios remotos 7
    3.1 Endpoints 7
    3.2 Encabezados, parámetros y respuestas 7
    3.3 Tratamiento de errores 8
4. Gestión de datos 9
    4.1 Modelo de datos 9
    4.2 Procesamiento de JSON 9
    4.3 Operaciones implementadas 9
5. Autenticación y autorización 11
    5.1 Inicio de sesión 11
    5.2 Administración de sesión 11
    5.3 Restricciones de acceso 11
6. Seguridad 13
    6.1 Medidas implementadas 13
    6.2 Análisis de riesgos 13
7. API de terceros: Open Food Facts 14
    7.1 Justificación 14
    7.2 Mecanismo de consumo 14
    7.3 Tratamiento de fallos 15
8. Pruebas y resultados 16
    8.1 Pruebas automáticas 16
    8.2 Pruebas funcionales en dispositivo físico 16
9. Avance funcional 18
    9.1 Relación de funcionalidades 19
10. Repositorio y entorno de desarrollo 20
11. Conclusiones 20
12. Referencias 21

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

1. Descripción del proyecto

1.1 Problemática

Los negocios pequeños suelen llevar su inventario en cuadernos u hojas de cálculo dispersas. Esto provoca capturas duplicadas, errores de conteo, productos agotados sin aviso y ausencia de un historial confiable de quién modificó las existencias. Además, registrar cada producto a mano (nombre, marca, categoría) consume tiempo.

1.2 Justificación

StockGo centraliza el inventario en un servicio remoto accesible desde el teléfono. El código de barras del producto se lee con la cámara y sus datos se consultan en una API pública (Open Food Facts), lo que reduce la captura manual. Los permisos se separan por rol para que un empleado solo registre movimientos y no pueda alterar el catálogo ni las cuentas.

1.3 Objetivo general

Desarrollar una aplicación móvil que permita gestionar el inventario de un negocio mediante un servicio remoto autenticado, con control de acceso por roles y consumo de una API de terceros para completar los datos de los productos.

1.4 Usuarios destinatarios

- **Administrador:** da de alta, edita y da de baja productos; crea y administra empleados; consulta el historial.
- **Empleado:** consulta el inventario y registra entradas y salidas de stock.

1.5 Funcionalidades principales

Funcionalidad	Descripción	Estado
Autenticación	Inicio de sesión con JWT; primer administrador por registro inicial	Implementada y probada
Roles	Administrador / Empleado con validación en servidor	Implementada y probada
Productos	Alta, consulta y baja lógica (administrador)	Implementada y probada
Movimientos	Entradas y salidas transaccionales con historial	Implementada y probada
Usuarios	Alta y activación/desactivación por administrador	Implementada y probada
Escáner	Lectura de código con la cámara (mobile_scanner)	Implementada y probada
API de terceros	Consulta en Open Food Facts por código de barras	Implementada y probada

Funcionalidad	Descripción	Estado
Edición de producto	Pantalla en la app (mantener pulsado un producto); usa PUT /products/<id>	Implementada; validar en dispositivo
Stock mínimo por producto	Alerta de stock bajo configurable (antes fijo en 5)	Implementada y probada con pytest
Filtros y orden del inventario	Stock bajo, categoría, A-Z y menor stock	Implementada; validar en dispositivo
Historial filtrable	Búsqueda, tipo y rango de fechas	Implementada; validar en dispositivo
Gráfica semanal	Entradas y salidas de los últimos 7 días (GET /stats/weekly)	Implementada y probada con pytest
Perfil de usuario	Cambiar nombre y contraseña (PUT /me, PUT /me/password)	Implementada y probada con pytest
Registro de acciones	Bitácora de altas, ediciones, bajas y exportaciones del administrador	Implementada y probada con pytest
Exportar inventario	CSV copiado al portapapeles (GET /products/export.csv)	Implementada y probada con pytest
Cola sin conexión	Los movimientos hechos sin red se envían al volver internet	Implementada; validar en dispositivo
Código interno de producto	Genera un código para productos sin código de barras	Implementada; validar en dispositivo
Límite de intentos de login	5 fallos bloquean 5 min (HTTP 429)	Implementada y probada con pytest
Modo sin conexión básico	Muestra el último inventario guardado cuando no hay red	Implementada; validar en dispositivo
Atribución a Open Food Facts	Texto con licencia ODbL en el formulario de producto	Implementada
Foto del producto con la cámara, APK con firma propia	Descritas en el backlog	Pendiente

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

2. Arquitectura

2.1 Componentes

La solución sigue una arquitectura cliente-servidor de tres capas. La aplicación Flutter nunca se conecta a la base de datos: toda operación pasa por la API REST, que valida el token, aplica las reglas de negocio y consulta MySQL. La API de Open Food Facts también se consume únicamente desde el servidor, de modo que el teléfono no depende de un tercero ni conoce sus detalles.

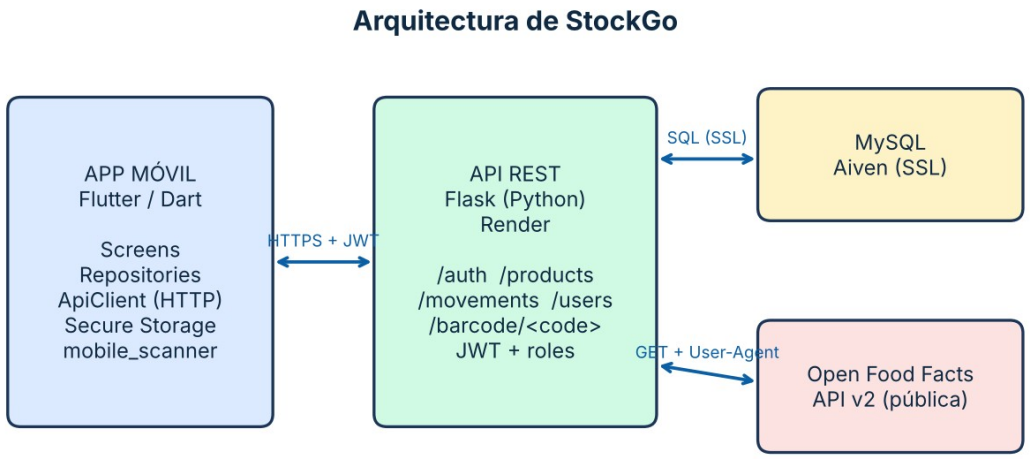


Figura 1. Componentes y comunicación de StockGo.

Componente	Tecnología	Alojamiento
Aplicación móvil	Flutter / Dart (Material 3)	APK en el dispositivo
API REST	Flask 3.0.3, PyJWT, gunicorn	Render (plan gratuito)
Base de datos	MySQL 8 (SSL)	Aiven (plan gratuito)
API de terceros	Open Food Facts API v2	Servicio público

2.2 Organización del código

```
StockGo/
├── backend/   app.py, requirements.txt, Procfile, .env.example
├── database/  schema.sql
├── mobile/lib/
│   ├── core/          api_client.dart, theme.dart, password_rules.dart
│   ├── repositories/  repositories.dart
│   ├── screens/       login, home, users, scanner
│   └── widgets/       peeking_dialog.dart (mascota)
├── tests/     test_basic.py
├── .github/workflows/ci.yml
└── docs/      documentación técnica
```

El cliente usa el patrón **Repository**: las pantallas no construyen peticiones HTTP, sino que llaman a repositorios que a su vez usan un único `ApiClient` (patrón Singleton) encargado del token, el tiempo de espera y la traducción

de errores.

2.3 Flujo de información

1) La pantalla invoca un repositorio. 2) `ApiClient` agrega el encabezado `Authorization: Bearer <token>` y envía la petición HTTPS. 3) La API valida el token, consulta de nuevo en la base el rol y el estado del usuario y ejecuta la operación con consultas parametrizadas. 4) La respuesta JSON se convierte en datos que la interfaz presenta con sus estados de carga, vacío, éxito o error.

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

3. Integración de servicios remotos

El servicio remoto propio es la API REST de StockGo, desplegada en Render y accesible en `https://stockgo-px32.onrender.com`. Todas las respuestas son JSON y los errores usan el formato `{"error": "mensaje"}`.

3.1 Endpoints

Método	Ruta	Rol	Propósito
GET	/health	Público	Verificar que el servicio responde
GET	/auth/has-users	Público	Indica si ya existe un administrador
POST	/auth/register	Público*	Crea el primer administrador (cerrado después)
POST	/auth/login	Público	Devuelve token JWT, rol y nombre
GET	/me	Autenticado	Datos del usuario en sesión
PUT	/me	Autenticado	Cambia el nombre
PUT	/me/password	Autenticado	Cambia la contraseña (pide la actual)
GET	/stats/weekly	Autenticado	Entradas y salidas de 7 días
GET	/audit	Admin	Registro de acciones del administrador
GET	/products/export.csv	Admin	Inventario en CSV
GET	/products	Autenticado	Lista productos activos
POST	/products	Admin	Crea un producto
PUT	/products/<id>	Admin	Edita un producto
DELETE	/products/<id>	Admin	Baja lógica (conserva historial)
POST	/movements	Autenticado	Entrada o salida de stock (transacción)
GET	/movements	Autenticado	Historial de movimientos
GET	/users	Admin	Lista usuarios
POST	/users	Admin	Crea un empleado o administrador
PUT	/users/<id>	Admin	Cambia rol, estado o contraseña
GET	/barcode/<code>	Admin	Consulta Open Food Facts

*El registro solo funciona mientras la tabla de usuarios esté vacía.

3.2 Encabezados, parámetros y respuestas

Elemento	Detalle
Encabezados de petición	`Content-Type: application/json` y `Authorization: Bearer <token>` (excepto rutas públicas)
Cuerpo de login	`{"email": "...", "password": "..."}`
Respuesta de login	`{"token": "...", "rol": "admin", "nombre": "..."}`
Cuerpo de movimiento	`{"product_id": 1, "tipo": "entrada", "cantidad": 10}`
Respuesta de movimiento	`{"ok": true, "stock": 18}` con código 201

3.3 Tratamiento de errores

Código	Causa	Mensaje / comportamiento en la app
400	Datos vacíos, correo o contraseña inválidos	Mensaje específico del campo faltante
401	Token ausente, vencido o cuenta desactivada	La app cierra la sesión y vuelve al login
403	Rol sin permiso	"No tienes permisos para esta acción"
404	Recurso inexistente	"Recurso no encontrado"
409	Stock insuficiente en una salida	"Stock insuficiente (N disponibles)"
429	Límite de intentos de login o de Open Food Facts	Mensaje con el tiempo de espera; el formulario sigue editable
502	Falla de Open Food Facts	Mensaje claro; el formulario sigue editable
Sin red	Tiempo de espera agotado (20 s; 60 s en la nube)	Se muestra el último inventario guardado con aviso de "Sin conexión"

`ApiClient` traduce estas situaciones a un tipo `ApiError` para que la interfaz muestre un mensaje comprensible sin exponer detalles internos. El plan gratuito de Render entra en reposo tras inactividad, por lo que la primera petición puede tardar cerca de 50 segundos; por eso la aplicación de producción se compila con `API_TIMEOUT=60`.

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

4. Gestión de datos

4.1 Modelo de datos

Tabla	Campos principales
usuarios	id, nombre, email (único), password_hash, rol (admin/empleado), activo, creado_en
productos	id, codigo_barras (único), nombre, marca, categoria, imagen_url, stock (CHECK ≥ 0), stock_minimo (CHECK ≥ 0), activo
movimientos	id, producto_id, usuario_id, tipo (entrada/salida), cantidad (CHECK > 0), fecha
auditoria	id, usuario_id, accion, entidad, detalle, fecha (solo inserciones)

Los movimientos solo se insertan, nunca se editan ni se borran, de modo que el historial es una bitácora fiable. Los productos eliminados solo se desactivan (baja lógica) para no romper las referencias del historial.

4.2 Procesamiento de JSON

El servidor lee el cuerpo con `request.get_json(silent=True)`, valida campos con la función `need()` y devuelve JSON con `jsonify`. En el cliente, `ApiClient` decodifica la respuesta con `dart:convert` y los repositorios la transforman en listas y mapas que consumen las pantallas. Ejemplo de respuesta de `/products`:

```
[{"id": 1, "codigo_barras": "7501055304745", "nombre": "Coca cola",
  "marca": "Coca-cola", "categoria": "Colas", "stock": 8}]
```

4.3 Operaciones implementadas

- **Consulta:** productos, movimientos y usuarios.
- **Registro:** productos (admin), usuarios (admin) y movimientos.
- **Actualización:** rol, estado y contraseña de usuarios; stock mediante movimientos.
- **Eliminación:** baja lógica de productos.

El movimiento se ejecuta en una transacción con bloqueo de fila, para que dos operaciones simultáneas no dejen el stock en negativo:

```
    try:  # transacción: stock + historial juntos
        cur.execute("SELECT stock FROM productos WHERE id=%s AND activo=1 FOR
UPDATE", (pid,))
        p = cur.fetchone()
        if not p:
            return jsonify(error="Producto no encontrado"), 404
        if d["tipo"] == "salida" and p["stock"] < qty:
            return jsonify(error=f"Stock insuficiente ({p['stock']} disponibles)"),
409
        delta = qty if d["tipo"] == "entrada" else -qty
        cur.execute("UPDATE productos SET stock=stock+%s WHERE id=%s", (delta, pid))
        cur.execute("INSERT INTO movimientos(producto_id,usuario_id,tipo,cantidad)
VALUES(%s,%s,%s,%s)",
                    (pid, g.user["id"], d["tipo"], qty))
        con.commit()
```

Fragmento de create_movement en backend/app.py.

5. Autenticación y autorización

5.1 Inicio de sesión

El usuario envía correo y contraseña a `POST /auth/login`. La API busca al usuario activo, compara la contraseña con `check_password_hash` y, si es correcta, emite un token JWT firmado con HS256 y 8 horas de vigencia que contiene el identificador y el rol. Si falla, responde siempre "Credenciales incorrectas", sin indicar si el correo existe.

5.2 Administración de sesión

La aplicación guarda el token, el rol y el nombre con `flutter_secure_storage` (almacenamiento cifrado del sistema), no en preferencias de texto plano. Al abrir la app, si hay token se entra directamente; si la API responde 401 en cualquier momento, `ApiClient` invoca un callback que borra la sesión y regresa al login mediante un `navigatorKey` global. El cierre de sesión borra los tres valores guardados.

5.3 Restricciones de acceso

Los permisos se aplican **en el servidor** con el decorador `@auth(roles)`. Además de validar la firma, vuelve a leer el rol y el estado desde la base de datos, de modo que si un administrador desactiva o cambia de rol a un usuario, el cambio aplica de inmediato aunque su token siga vigente. La interfaz solo oculta opciones por comodidad; no es la barrera de seguridad.

Acción	Administrador	Empleado	Sin sesión
Ver inventario e historial	Sí	Sí	No
Registrar entrada/salida	Sí	Sí	No
Crear/editar/eliminar producto	Sí	No (403)	No (401)
Consultar Open Food Facts	Sí	No (403)	No (401)
Crear y administrar usuarios	Sí	No (403)	No (401)

```
def auth(roles=None):
    def deco(f):
        @functools.wraps(f)
        def wrapper(*a, **k):
            token = request.headers.get("Authorization", "").removeprefix("Bearer ")
            try:
                p = jwt.decode(token, SECRET, algorithms=["HS256"])
            except Exception:
                return jsonify(error="Token inválido o expirado"), 401
            if roles and p["rol"] not in roles:
                return jsonify(error="No tienes permisos para esta acción"), 403
            # El rol y el estado se leen de la base: un cambio del admin aplica de
inmediato
            u = query("SELECT id,rol,activo FROM usuarios WHERE id=%s", (p["id"],),
one=True)
            if not u or not u["activo"]:
                return jsonify(error="Tu cuenta está desactivada"), 401
            if roles and u["rol"] not in roles:
                return jsonify(error="No tienes permisos para esta acción"), 403
            g.user = {"id": u["id"], "rol": u["rol"]}
```

Figura 2. Decorador de autorización por rol.

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

6. Seguridad

6.1 Medidas implementadas

Medida	Implementación
Contraseñas	Hash con Werkzeug (`generate_password_hash`); nunca se guardan ni se devuelven en texto plano
Política de contraseña	Mínimo 8 caracteres, un número y un carácter especial; validada en servidor (`validar_password`) y en la app (checklist en vivo)
SQL Injection	Todas las consultas usan parámetros (`%s`)
Secretos	`SECRET_KEY`, credenciales de base de datos y certificado se leen de variables de entorno; `.env` está en `.gitignore` y se incluye `.env.example` sin valores
Token en el dispositivo	`flutter_secure_storage`
Transporte	HTTPS hacia Render; conexión SSL hacia MySQL en Aiven
Errores	Mensajes genéricos al usuario; no se devuelven trazas ni consultas
Primer administrador	El registro público se cierra cuando existe un usuario
Integridad	CHECK en stock y cantidad; transacción con bloqueo de fila; historial inmutable
Auditoría	Las acciones del administrador se registran en una bitácora; un fallo de registro no interrumpe la operación
Exportación CSV	Se neutralizan celdas que comienzan con `=`, `+`, `-` o `@` (inyección de fórmulas)
Pruebas automáticas	50 pruebas de pytest ejecutadas en GitHub Actions en cada push

6.2 Análisis de riesgos

Riesgo	Situación actual	Mitigación / pendiente
Fuerza bruta en login	Bloqueo de 5 min tras 5 intentos fallidos por IP y correo (HTTP 429)	El contador está en memoria y se reinicia si el servidor reinicia
Robo de token	Vigencia de 8 h; guardado cifrado	Re-validación de rol/estado en cada petición
Fuga de secretos	Variables de entorno; `.env` ignorado	Rotar claves si se expusieran
Certificado SSL de BD	Con `DB_SSL_CA` se verifica el certificado; sin él se omite y el log lo avisa	**Pendiente:** configurar `ca.pem` y `DB_SSL_CA` en Render
Abuso de la API de terceros	Ruta solo para administrador; error 429 controlado	Caché de 24 horas para consultas repetidas
APK sin firma de producción	Se instala en modo release con clave de depuración	**Pendiente:** firma con keystore propio

7. API de terceros: Open Food Facts

7.1 Justificación

Open Food Facts es una base de datos colaborativa y abierta de productos alimenticios con más de dos millones de registros, gratuita y sin clave de acceso. Se eligió porque devuelve nombre, marca, categoría e imagen a partir del código de barras, justo los datos que se capturaban a mano al dar de alta un producto. Su limitación es que no contiene todos los productos (especialmente artículos no alimenticios o locales).

7.2 Mecanismo de consumo

Elemento	Detalle
Documentación	https://openfoodfacts.github.io/openfoodfacts-server/api/
Endpoint	`GET https://world.openfoodfacts.org/api/v2/product/{codigo}.json`
Parámetro	`fields=code,product_name,brands,categories,image_front_url`
Autenticación	Ninguna. Se exige un `User-Agent` identificable: `StockGo/1.0 (proyecto escolar)`
Límite de uso	Lectura de producto: 15 peticiones por minuto por IP (según el proveedor)
Llamada desde	El servidor Flask (`/barcode/<code>`), no directamente desde la app

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

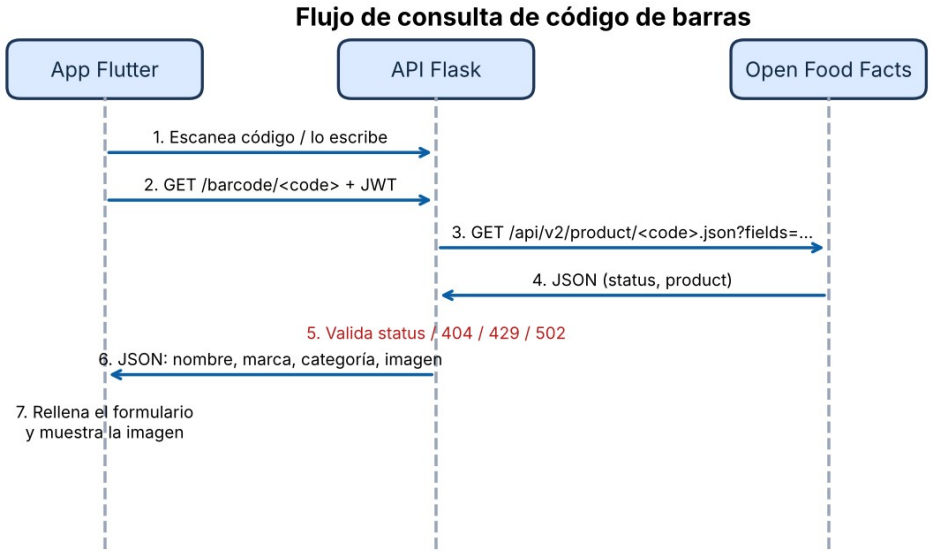


Figura 3. Flujo de una consulta de código de barras.

```
    if not code.isdigit() or not 8 <= len(code) <= 14:
        return jsonify(error="Código de barras inválido"), 400
    hit = _off_cache.get(code)
    if hit and time.time() - hit[0] < OFF_TTL:  # no gasta el límite de 15 consultas
por minuto
        return jsonify(hit[1])
    try:
        r = requests.get(
            OFF_URL.format(code),
            params={"fields": OFF_FIELDS},
            headers={"User-Agent": "StockGo/1.0 (proyecto escolar)"},
            timeout=8,
        )
    except requests.RequestException:
        return jsonify(error="No se pudo contactar Open Food Facts"), 502
    if r.status_code == 429:  # límite de consultas del proveedor
        return jsonify(error="Open Food Facts limitó las consultas. "
                             "Intenta de nuevo en un minuto."), 429
    if r.status_code >= 500:
        return jsonify(error="Open Food Facts no está disponible"), 502
    try:
        data = r.json()
    except ValueError:
        return jsonify(error="Respuesta inválida de Open Food Facts"), 502
```

Figura 4. Fragmento de la ruta de consulta de código de barras en backend/app.py.

7.3 Tratamiento de fallos

Situación	Código	Respuesta
Código no numérico o longitud inválida	400	"Código de barras inválido" (no se consulta al proveedor)
Producto inexistente (`status` ≠ 1)	404	"Producto no encontrado en Open Food Facts"
Límite de consultas del proveedor	429	Mensaje para reintentar en un minuto
Falla o caída del proveedor / respuesta inválida	502	"No se pudo contactar…" / "no está disponible"
Sin respuesta	Tiempo de espera de 8 s	Error 502 controlado

En todos los casos el formulario de producto permanece abierto para que el administrador capture los datos manualmente.

La interfaz muestra la atribución a Open Food Facts (la base se distribuye bajo licencia ODbL) en el formulario de producto. Las consultas repetidas se guardan en caché 24 horas para no gastar el límite del proveedor.

8. Pruebas y resultados

8.1 Pruebas automáticas

Se ejecutan con `pytest tests -v` y en GitHub Actions en cada push. Resultado obtenido: **50 pruebas aprobadas** (7 básicas de autenticación y roles; 22 de límite de intentos, movimientos de inventario y errores de Open Food Facts; y 21 de perfil, stock mínimo, auditoría, exportación CSV, estadísticas y caché de Open Food Facts).

Prueba	Esperado	Obtenido
Bloqueo tras 5 intentos fallidos de login	429 con Retry-After	Aprobada
Login correcto reinicia el contador	200	Aprobada
Salida mayor al stock	409, stock sin cambios	Aprobada
Dos salidas seguidas (6 + 6 con stock 10)	201 y 409; stock 4	Aprobada
Movimiento con datos inválidos (5 casos)	400	Aprobada
Open Food Facts: 429, 503, JSON inválido, producto inexistente, timeout, sin red	429 / 502 / 404	Aprobada
Cambio de contraseña propia (correcta y 3 rechazos)	200 / 400	Aprobada
Stock mínimo válido e inválido	Valor o error 400	Aprobada
Exportación CSV neutraliza fórmulas; empleado recibe 403	CSV seguro / 403	Aprobada
Estadísticas semanales rellenan 7 días	7 elementos	Aprobada
Auditoría: registro, 403 a empleados y falla tolerada	201 / 403	Aprobada
Caché de Open Food Facts: reutiliza y expira	1 y 2 llamadas externas	Aprobada
Detección de campos vacíos (need)	Reporta faltantes	Aprobada
Ruta protegida sin token	401	Aprobada
Empleado crea producto	403	Aprobada
Empleado administra usuarios	403	Aprobada
Empleado elimina producto	403	Aprobada
Contraseña válida	Sin error	Aprobada
Contraseña inválida	Indica qué falta	Aprobada

La concurrencia real entre dos usuarios simultáneos no se puede reproducir sin una base de datos; las pruebas verifican que la consulta use `FOR UPDATE` y que dos salidas seguidas nunca dejen el stock negativo.

8.2 Pruebas funcionales en dispositivo físico

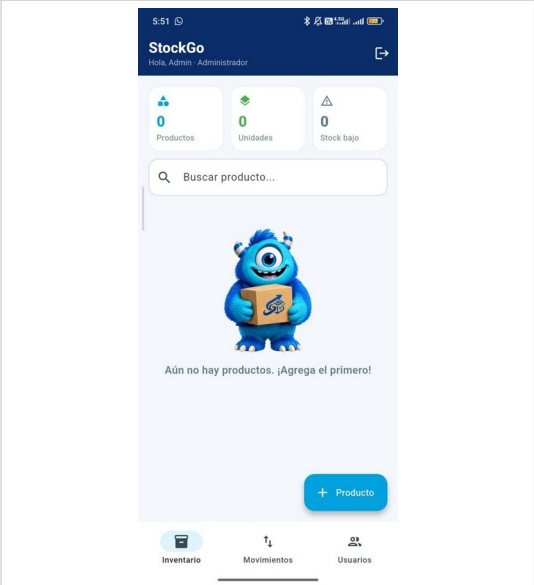
Procedimiento: APK de producción instalado en un teléfono Android conectado a la API en Render y a MySQL en Aiven.

#	Caso	Esperado	Obtenido
1	Inventario sin productos	Estado vacío con mascota	Correcto (captura 1)
2	Consultar código 7501055304745	Se rellenan nombre, marca, categoría e imagen	Correcto (captura 6)
3	Guardar producto escaneado	Aparece en inventario con su imagen	Correcto (captura 7)
4	Consultar producto inexistente / ruta inexistente	Mensaje de error visible	"Recurso no encontrado" (captura 8)
5	Registrar entradas y salidas	Stock actualizado y movimientos en el historial	Correcto (captura 3)
6	Alta de empleado con contraseña débil	Checklist indica lo que falta; no se permite crear	Correcto (captura 5)
7	Lista de usuarios y roles (admin)	Muestra administradores y empleados	Correcto (captura 4)
8	Empleado intenta crear producto (API)	403	Aprobada por prueba automática
9	Backend dormido en Render	La app espera y responde	Correcto con tiempo de espera de 60 s
10	Límite 429 de Open Food Facts	Mensaje de reintento	**No probado en dispositivo**; solo implementado
11	Bloqueo tras 5 intentos fallidos de login	429 y mensaje de espera	Aprobada por pytest; **no probado en dispositivo**
12	Editar un producto desde la app	Datos actualizados en el inventario	**Pendiente de validar en dispositivo**
13	Abrir la app sin internet	Último inventario guardado con aviso	**Pendiente de validar en dispositivo**
14	Registrar un movimiento sin internet	Se guarda y se envía al volver la red	**Pendiente de validar en dispositivo**
15	Filtros, perfil, exportación y registro de acciones	Funcionan en el teléfono	**Pendiente de validar en dispositivo**

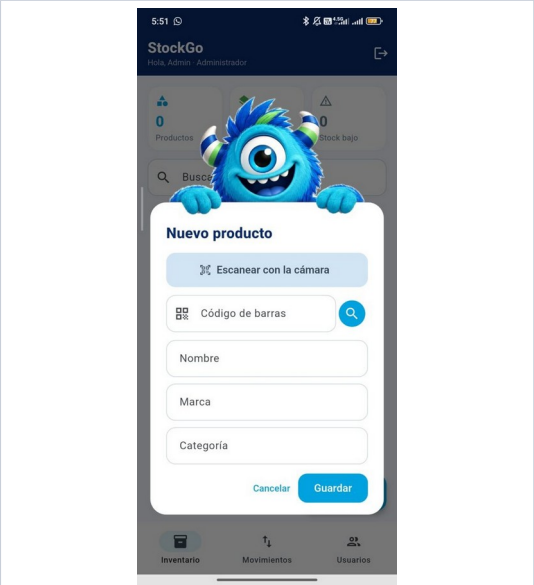
Limitaciones identificadas: la primera petición tras inactividad en Render tarda hasta ~50 s; no hay funcionamiento sin conexión; Open Food Facts no contiene todos los productos.

9. Avance funcional

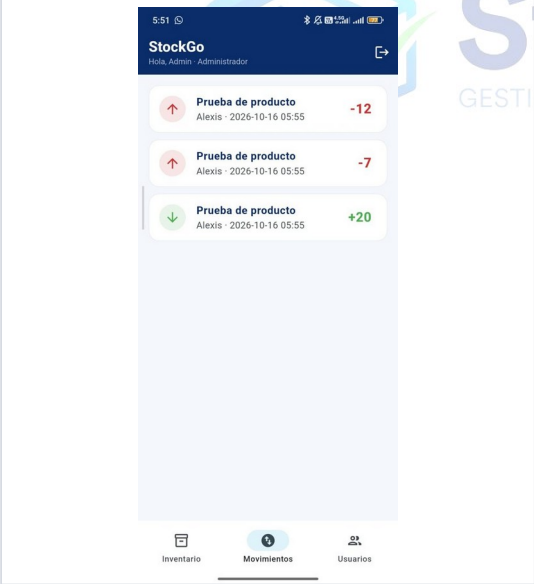
Capturas tomadas del dispositivo con la aplicación en ejecución conectada a servicios reales.



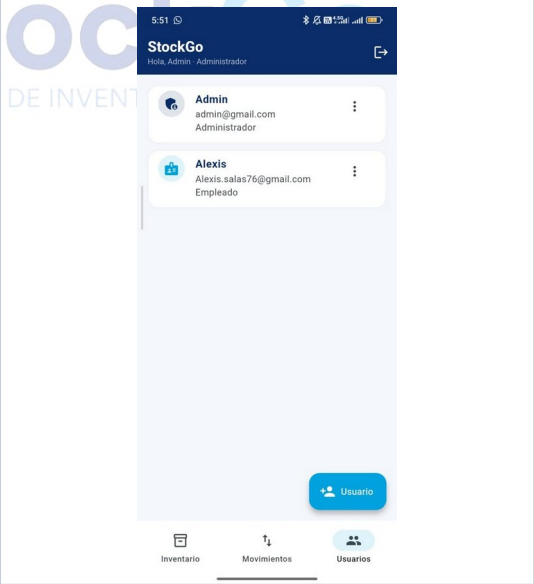
Captura 1. Inventario vacío



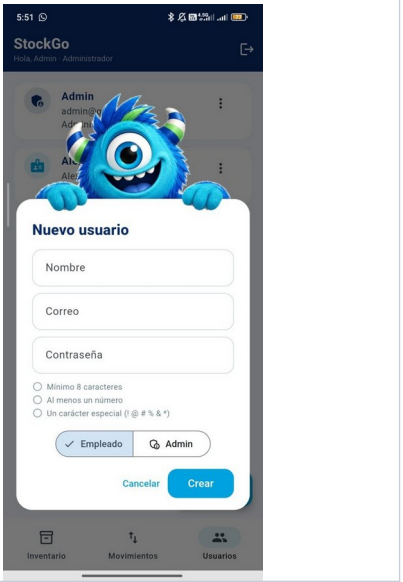
Captura 2. Formulario de nuevo producto



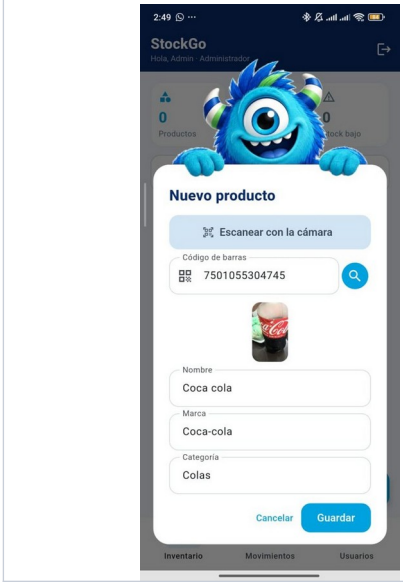
Captura 3. Historial de movimientos



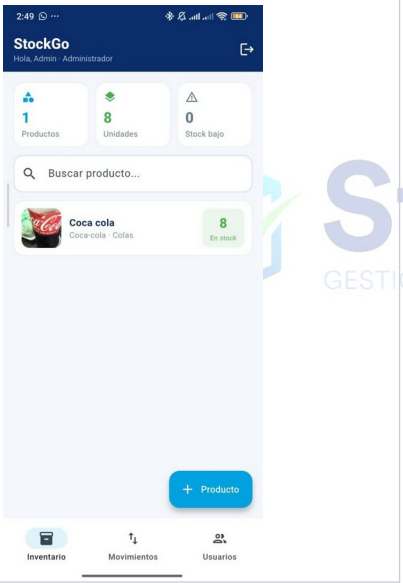
Captura 4. Gestión de usuarios (admin)



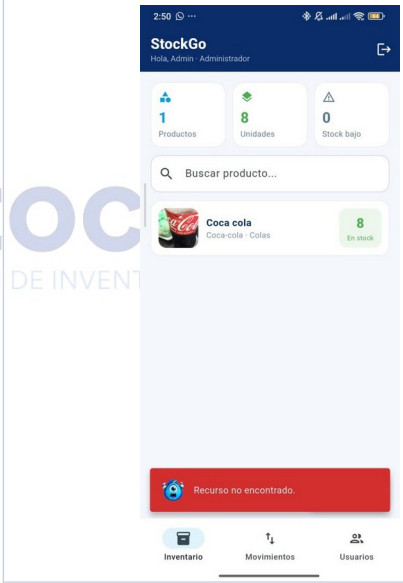
Captura 5. Alta de usuario con validación de contraseña



Captura 6. Datos obtenidos de Open Food Facts



Captura 7. Inventario con producto e imagen



Captura 8. Manejo de error de la API

9.1 Relación de funcionalidades

Funcionalidad	Estado	Evidencia
Autenticación y sesión	Implementada y probada	Capturas 1–8; pruebas 401/403
Roles Admin/Empleado	Implementada y probada	Captura 4; pruebas automáticas
Productos y escáner	Implementada y probada	Capturas 2, 6, 7
Movimientos de stock	Implementada y probada	Captura 3

Funcionalidad	Estado	Evidencia
API de terceros	Implementada y probada	Captura 6
Edición de producto desde la app	Implementada; validar en dispositivo	Pantalla de inventario
Stock mínimo, perfil, auditoría, exportación y estadísticas	Implementadas y probadas (pytest)	Pruebas automáticas
Filtros, historial filtrable, gráfica, código interno y cola sin conexión	Implementadas; validar en dispositivo	Código de la app
Límite de intentos de login	Implementada y probada (pytest)	Pruebas automáticas
Modo sin conexión básico	Implementada; validar en dispositivo	Caché del último inventario
Atribución a Open Food Facts en la interfaz	Implementada	Formulario de producto
Foto del producto con la cámara	Pendiente	—
APK firmado con clave propia	Pendiente (guía en docs/firmar_apk.md)	—

10. Repositorio y entorno de desarrollo

- Repositorio: https://github.com/alexis20051212/StockGo
- Ramas: `main` (despliega en Render), `develop` y `feature/*`.
- Commits con formato `tipo: descripción` (feat, fix, docs, test, ci).
- `README.md` con requisitos, instalación, variables de entorno, ejecución y pruebas.
- `.gitignore` excluye `.env`, `build/`, `.dart_tool/` y llaves de firma; `.env.example` documenta las variables sin valores.
- Integración continua: GitHub Actions ejecuta las 50 pruebas en cada push y pull request.

11. Conclusiones

Se logró una aplicación operativa de extremo a extremo: interfaz Flutter, API propia en la nube, base de datos remota, autenticación con roles y consumo real de Open Food Facts. Las pruebas automáticas y las capturas del dispositivo respaldan lo declarado.

Dificultades: configuración del entorno de desarrollo (SDK, NDK, ADB), despliegue en Render (versión de Python y variables de entorno), y el reposo del plan gratuito, que obligó a ampliar el tiempo de espera.

Mejoras pendientes: foto del producto con la cámara (requiere almacenamiento de imágenes), APK firmado con clave propia, configuración

del certificado SSL de la base de datos (`ca.pem`) y validación en dispositivo de las funciones nuevas de la aplicación (edición, filtros, perfil, exportación, registro de acciones y cola sin conexión).

12. Referencias

- Flutter. Documentation. https://docs.flutter.dev
- Dart. Language documentation. https://dart.dev/guides
- Flask. Documentation. https://flask.palletsprojects.com
- PyJWT. Documentation. https://pyjwt.readthedocs.io
- Werkzeug. Security utilities. https://werkzeug.palletsprojects.com
- MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
- Open Food Facts. API documentation. https://openfoodfacts.github.io/openfoodfacts-server/api/
- Render. Documentation. https://render.com/docs
- Aiven. MySQL documentation. https://aiven.io/docs/products/mysql
- GitHub. GitHub Actions documentation. https://docs.github.com/actions
- mobile_scanner. pub.dev package. https://pub.dev/packages/mobile_scanner
- flutter_secure_storage. pub.dev package. https://pub.dev/packages/flutter_secure_storage

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE